

Q1.Explain the heuristic function.-A heuristic function (denoted as $h(n)$) is an **estimate of the cost or distance from the current node 'n' to the goal state**.It helps in deciding **which nodes to explore** and **when to stop** exploring unpromising paths.**Heuristic Function:** A function that provides an approximate measure of how close a given state is to the goal state. ♦ **Purpose of Using Heuristic Function for Cutoff**-The cutoff in search is used to **limit the depth or cost** of exploration when the search space becomes too large or time-consuming.The heuristic function helps:**Reduce computational cost** by pruning unpromising nodes.**Prevent infinite searches** in problems with large or infinite state spaces.**Focus the search** on likely solution paths. **Common Types of Heuristic Cutoff Strategies**-**1. Depth-Limited Search (DLS)**A cutoff depth 'l' is defined.The search stops (cuts off) when the depth = l, even if the goal isn't found.Prevents infinite loops but may miss the solution if it lies beyond the cutoff depth.**2. Iterative Deepening Search (IDS)**-Repeatedly applies depth-limited search with increasing cutoff limits.Ensures optimality while avoiding infinite search paths.**3. Heuristic Cutoff in A* or Best-First Search**-In A* and **Best-First Search**, heuristics are used to decide **which nodes to expand** and **which to prune**. ▫ **Best-First Search** Uses $h(n)$ = estimated cost to goal Expands the node with the **lowest heuristic value** first.Nodes with high heuristic values (less promising) are **cut off**. ▫ **A* Search**-Uses the **evaluation function**: $f(n) = g(n) + h(n)$ where, $g(n)$: cost so far from the start node to 'n'. $h(n)$: estimated cost from 'n' to the goal.The algorithm prunes nodes whose $f(n)$ exceeds the best known cost to the goal → **cutoff using heuristic**. ♦ **Cutoff Function Example**-In **Alpha-Beta Pruning** (used in game trees), a heuristic cutoff is applied to avoid unnecessary exploration:**Evaluation function** ($f(n)$) estimates the utility (win/lose likelihood).Search is **cut off at a certain depth**, and the **heuristic evaluation** is used instead of the actual terminal value.This method reduces the number of nodes explored drastically. ♦ **Properties of a Good Heuristic Function**-A heuristic function should have the following properties:**Admissible**- Never overestimates the cost to reach the goal (ensures optimality). **Consistent (Monotonic)**-For every node n and successor n': $h(n) \leq c(n, n') + h(n')$ **Computationally Efficient**- Should be easy and fast to compute. **Problem-Specific**- Should utilize domain knowledge effectively.**Advantages of Heuristic Cutoff**-✅ Reduces search time and memory usage ✅ Prevents infinite search loops ♦ **Disadvantages**-❌ May miss the optimal solution if heuristic is not admissible ❌ Depends heavily on quality of heuristic function.

Q1c.List all problem-solving strategies. 🧠 **1. Problem Solving Strategies in AI**-Problem-solving strategies are **systematic approaches** used by AI agents to find solutions efficiently.The common strategies include:
List of Problem-Solving Strategies-Generate and Test,Hill Climbing,Best-First Search,Means–Ends Analysis ,Constraint Satisfaction,Backtracking,Branch and Bound,A* Search Algorithm,Goal Stack Planning,Heuristic Search 🌱 **2. What is Backtracking?**-Backtracking is a **depth-first search strategy** used to **find solutions incrementally by exploring all possible options**.When a partial solution violates a constraint, the algorithm **backtracks (goes one step back)** and tries another option. ♦ It is often described as **“trial and error with pruning.”** ♦ **Steps in Backtracking**-Choose a possible solution (make a move). Check if it is **safe (valid)**.If valid → proceed to next step.If invalid → **undo (backtrack)** and try another option.Repeat until all constraints are satisfied or no solution exists. 👑 **3. Example – N-Queen Problem** 🧠 **Problem Statement:**Place N queens on an N×N chessboard so that **no two queens attack each other**, i.e.,No two queens share the same row, column, or diagonal. ♦ **Backtracking Approach for N-Queen** **Algorithm Steps:**Place a queen in the first column of the first row.Move to the next row and try to place a queen in a safe column.If no safe column exists → backtrack to the previous row and move the queen to the next safe column.Repeat until all queens are placed safely.If a solution is found, print the board.

Q3a. Explain FOL inference for the following quantifiers:

1) Universal Generalization (UG): If a statement is true for every individual in the domain, then we can generalize it using a **universal quantifier ($\forall$)**. **Rule Form:** If $P(x)$ is true for all x in the domain, then we can infer $\forall x P(x)$. **Example:** If we have proved that $P(\text{John})$, $P(\text{Mary})$, $P(\text{Alice})$ are true for all individuals, then we can write: $\forall x P(x)$. **Explanation:** This means we generalize from **specific instances to a universal truth**.

2) Universal Instantiation (UI): If a statement is known to be true for **all members of a domain**, then it is true for **any specific individual** in that domain. **Rule Form:** From $\forall x P(x)$, we can infer $P(c)$, where c is any constant in the domain. **Example:** Given: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ We can infer: $\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$. **Explanation:** This allows us to take a **universal statement** and apply it to a **specific case**.

3) Existential Instantiation (EI): If there exists at least one individual for which a statement holds true, we can replace the existential variable with a **new constant symbol (not appearing elsewhere)**. **Rule Form:** From $\exists x P(x)$, we can infer $P(c)$, where c is a new constant representing that "some object" exists. **Example:** Given: $\exists x (\text{Cat}(x) \wedge \text{Black}(x))$ We can infer: $\text{Cat}(a) \wedge \text{Black}(a)$, where a denotes that particular cat. **Explanation:** We instantiate the existential quantifier to a specific but **unknown individual**.

4) Existential Introduction (EI' or EI2): If we know that a statement is true for a **specific object**, then we can conclude that **there exists** some object for which the statement holds. **Rule Form:** From $P(c)$, we can infer $\exists x P(x)$. **Example:** Given: $\text{Bird}(\text{Tweety})$, we can infer: $\exists x \text{Bird}(x)$. **Explanation:** We move from a **specific instance to an existential statement**, indicating that *at least one* such object exists.

Q3c. Forward Chaining- Forward chaining is a **data-driven inference technique** where reasoning starts from **known facts** and applies inference rules to **derive new facts** until the **goal (conclusion)** is reached. It works from **facts $\rightarrow$ conclusion**. **Procedure / Steps:** Start with the **initial set of known facts** in the knowledge base. **Match** these facts with the **IF** parts (antecedents) of rules. When all conditions of a rule are satisfied, **fire** the rule and **add** its conclusion (THEN part) as a new fact. Repeat until the **goal** or **no new facts** can be inferred. **Properties:** **Direction-** Data-driven (from facts to goal). **Control Strategy-** Forward movement through rules. **Application Area-** Expert systems, sensor data analysis, diagnostic systems. **Termination-** Stops when no new rules fire or when goal is achieved. **Example: Knowledge Base:** IF it rains $\rightarrow$ ground is wet. IF ground is wet $\rightarrow$ grass is slippery. It rains (fact) **Process:** Step 1: Fact "It rains" matches rule 1 $\rightarrow$ infer "ground is wet". Step 2: Now "ground is wet" matches rule 2 $\rightarrow$ infer "grass is slippery".

Backward Chaining- Backward chaining is a **goal-driven inference technique** where reasoning starts from a **goal (hypothesis)** and works **backward** to determine what facts must be true to support that goal. It works from **goal $\rightarrow$ facts**. **Procedure / Steps:** Start with a **goal (what you want to prove)**. Search for rules whose **THEN part (conclusion)** matches the goal. For each such rule, try to prove its **IF part (premises)**. If a premise is not known, **set it as a new sub-goal** and attempt to prove it using other rules or known facts. Continue until: All sub-goals are proven (goal achieved), or No rule/fact supports the goal (failure). **Properties:** **Direction-** Goal-driven (from goal to facts). **Control Strategy-** Works backward through rules. **Application Area-** Diagnosis, planning, query systems (e.g., Prolog). **Termination-** Stops when goal is proven or disproven. **Example: Knowledge Base:** IF it rains $\rightarrow$ ground is wet . IF ground is wet $\rightarrow$ grass is slippery . It rains (fact) **Goal:** Is the grass slippery? **Process:** Step 1: To prove "grass is slippery", check rule 2 $\rightarrow$ requires "ground is wet". Step 2: To prove "ground is wet", check rule 1 $\rightarrow$ requires "it rains". Step 3: Fact "it rains" is known $\rightarrow$ hence "ground is wet" true $\rightarrow$ "grass is slippery" true.

Q.4a.(i) State Space Search Planning-Idea: Plan is formed by searching through the **space of all possible world states**. Two main types: **Forward State-Space Search** (Progression) **Backward State-Space Search** (Regression) → **Forward Search:** Start from the **initial state**, apply actions to reach the **goal state**. Example: STRIPS planner. Used when the number of actions is limited. → **Backward Search:** Start from the **goal**, find actions that can produce it, then recursively find preconditions of those actions. More efficient when goals are specific. **Pros:** Simple and systematic. **Cons:** May explore large state spaces → computationally expensive.

(ii) Plan Space (Partial Order) Planning Idea: Instead of planning in terms of states, this approach searches through the **space of partial plans**. A partial plan specifies: Some actions, Some orderings, Some variable bindings. The planner gradually **refines** the plan until it becomes **complete** (totally ordered). **Example: POP (Partial-Order Planner)** — allows actions to be unordered when possible (parallel execution). **Pros:** Efficient, supports parallelism. **Cons:** Complex to implement.

(iii) Goal Stack Planning Idea: Used in early systems like **STRIPS**. A **stack** is used to keep track of **goals and subgoals**. **Process:** Push the **main goal** onto the stack. Decompose it into **subgoals**. Push the subgoals and required actions onto the stack. Continue until all subgoals are achieved. **Pros:** Simple implementation. **Cons:** Cannot handle interdependent goals effectively.

(iv) Hierarchical Planning (HTN – Hierarchical Task Network)-Idea: Decompose complex goals into smaller, simpler **subtasks** or **primitive actions**. **Process:** Start with a **high-level task**. **Decompose** it recursively into smaller subtasks. Continue until all tasks are **primitive (executable)**. **Example:** “Plan a trip” → “Book tickets”, “Pack luggage”, “Go to airport”. **Pros:** Mirrors human reasoning; efficient for large, structured problems. **Cons:** Requires pre-defined hierarchy of tasks.

(v) Conditional Planning-Idea: Used when the environment is **uncertain** or **partially observable**. The plan includes **conditional branches** based on possible outcomes of actions. **Example:** “If the weather is clear → go by car, else → stay home.” **Pros:** Handles uncertainty and dynamic conditions. **Cons:** Complex due to multiple branches.

(vi) Probabilistic / Reactive Planning-Probabilistic Planning: Actions have **probabilistic outcomes**. Uses **Markov Decision Processes (MDPs)** or **Partially Observable MDPs (POMDPs)**. **Reactive Planning:** System reacts in **real time** to environmental changes. No precomputed plan; uses **situation–action rules**. **Example:** Autonomous robots using sensors to avoid obstacles dynamically.

Q4b. Explain AI Architecture with a suitable diagram.—An AI Architecture is the conceptual framework that defines how different components of an Artificial Intelligence system work together to perform intelligent tasks such as learning, reasoning, perception, and decision-making. It provides the structure and flow of how input data is processed into intelligent output.

2. Basic AI System Architecture=An AI system can be divided into four main components: Perception (Input Layer). Knowledge Base (Memory / Storage) . Inference Engine (Reasoning / Decision Unit) . Action (Output Layer)

3. Components in Detail

(i) Perception System—The input stage of an AI system. Collects raw data from the environment through sensors, cameras, or other sources. Converts unstructured data (like text, images, audio) into a machine-understandable format. Example: Voice input in Alexa, camera input in self-driving cars.

(ii) Knowledge Base Acts as the memory of the AI system. Stores facts, rules, ontologies, and relationships between entities. Used by the inference engine for decision-making. Example: In Expert Systems, the knowledge base contains IF-THEN rules. In ML models, it stores trained parameters.

(iii) Inference Engine—The “brain” of the AI system. Applies logical reasoning or machine learning algorithms on the knowledge base to infer conclusions or make decisions. Implements techniques such as: Forward chaining / Backward chaining. Heuristic search / Neural inference . Example: Decides if a patient has a disease based on symptoms and stored rules.

(iv) Learning Component (Optional in modern AI systems)—Enables the system to learn from experience or data. Updates the knowledge base dynamically using learning algorithms (e.g., supervised, unsupervised, reinforcement learning).

(v) Action / Actuator—Converts the inference or decision into real-world action or output. Can be a display message, robot movement, voice output, etc.

Q2a-In Artificial Intelligence, **Knowledge Representation (KR)** is the process of **storing information about the world** in a form that a computer system can utilize to **solve complex problems**, such as diagnosing diseases, understanding language, or reasoning logically. The main goal of KR is to make knowledge **understandable to machines** while maintaining **logical reasoning capability**.

2. Knowledge Representation Using Logic Logic is one of the **most common and powerful methods** for representing knowledge in AI. Two major logic systems are:

1. Propositional Logic (PL) **2. Predicate Logic (First Order Logic – FOL)** ❄️ **3. Knowledge Representation Using Propositional Logic-** Propositional Logic (also called **Sentential Logic**) represents **facts as statements or propositions** that are either **True (T)** or **False (F)**. A **proposition** is a declarative statement that can have one of two truth values — True or False. ❖ **Syntax (Building Blocks):**

1) Proposition Symbols- Represent facts using letters. P, Q, R, etc. **2) Logical Connectives-** Combine or modify propositions. ¬ (NOT), ∧ (AND), ∨ (OR), → (IMPLIES), ↔ (EQUIVALENT) **3) Parentheses-** Define precedence. (P ∧ Q) → R ❖ **Semantics:** Defines the **truth value** of complex expressions based on their atomic propositions. Example: If P: "It is raining" → True Q: "The ground is wet" → True Then: (P → Q) means "If it is raining, then the ground is wet." → **True** ❖ **4. Example of Knowledge Representation using Propositional Logic-** It is raining-P The ground is wet-Q Thus, knowledge about the environment can be expressed as a **set of logical statements (KB – Knowledge Base)**.

5. Inference Rules in Propositional Logic Inference allows the system to **derive new facts** from existing ones.

Rule	Description	Example
Modus Ponens	If $P \rightarrow Q$ and P are true, then Q is true	$(P \rightarrow Q), P \Rightarrow Q$
Modus Tollens	If $P \rightarrow Q$ and ¬Q are true, then ¬P is true	$(P \rightarrow Q), \neg Q \Rightarrow \neg P$
AND Introduction	If P and Q are true, then (P ∧ Q) is true	$P, Q \Rightarrow (P \wedge Q)$
Resolution	Used in automated reasoning systems	From $(P \vee Q), (\neg Q \vee R) \Rightarrow (P \vee R)$

❖ **6. Limitations of Propositional Logic** -Cannot represent relationships between objects. Cannot handle variables or quantifiers. Not suitable for complex real-world knowledge.

Aspect	Propositional Logic (PL)	Predicate Logic (First Order Logic - FOL)
Basic Unit	Proposition (statement)	Predicate (relation) with variables
Expressiveness	Limited	Highly expressive
Variables	Not supported	Supported (x, y, etc.)
Quantifiers	Not available	Uses ∀ (for all), ∃ (there exists)
Example Representation	"It is raining" → P	"Rains(City)"
Knowledge Type	Simple facts	Relations between objects
Inference Power	Low	High